МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)
Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Обработка последовательностей

Студент
Гр. БИН-26-03 _____ М. П. Гришин
Студент
Гр. БИН-26-03 _____ Т. Н. Бобров
Преподаватель _____ М. В. Водяницкий

Владивосток 2026

1 Ход работы

Цель работы – изучить применение циклических конструкций в языке Python и научиться составлять программы для обработки последовательностей входных данных без сохранения их в коллекциях.

1.1 Задание 1. Обратный отсчёт

Пользователь вводит неотрицательное целое число n. С помощью цикла while необходимо вывести все целые числа от n до 0 включительно в порядке убывания, каждое на отдельной строке.

Блок-схема алгоритма программы для задания 1 представлена на рисунке 1.

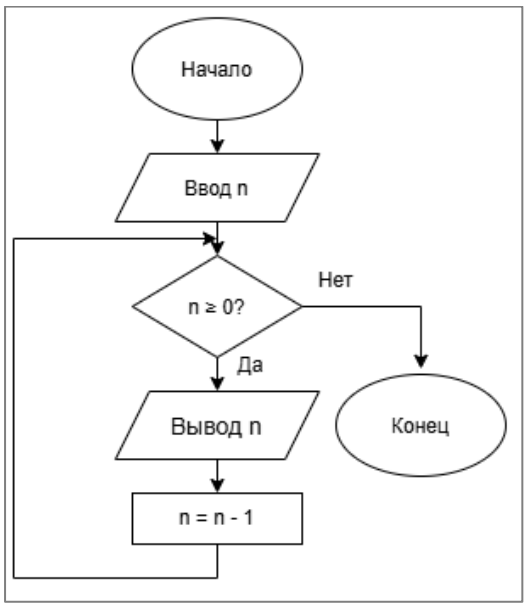


Рисунок 1 - Блок-схема алогоритма для задания 1

Текст программы для задания 1 представлен на рисунке 2.

```
n = int(input())
while n >= 0:
    print(n)
    n -= 1
```

Рисунок 2 – Текст программы для задания 1

Входные данные: 3

Результат: 3 2 1 0

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.

1.2 Задание 2. Сумма и произведение

Пользователь вводит целые числа a и b на отдельных строках, причём a ≤ b. С помощью цикла for необходимо вычислить сумму и произведение всех целых чисел от a до b включительно.

Блок-схема алгоритма программы для задания 2 представлена на рисунке 3.

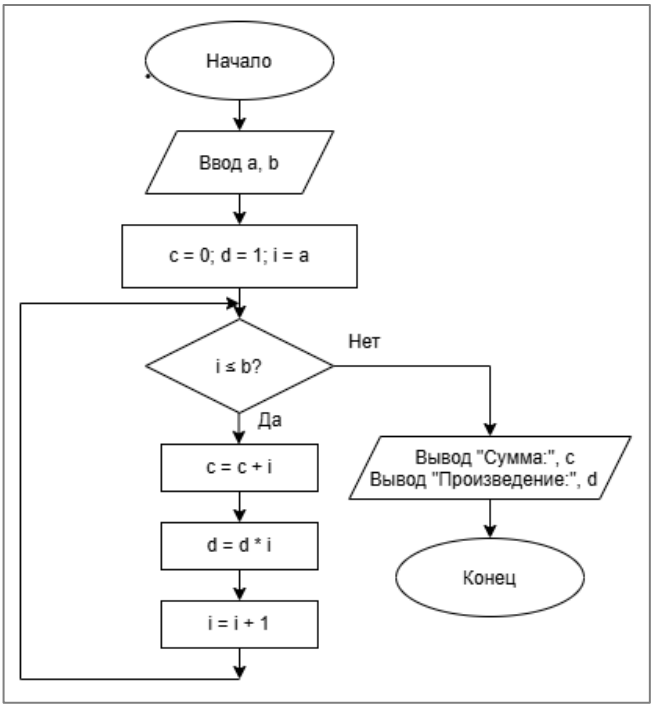


Рисунок 3- Блок-схема алгоритма для задания 2

Текст программы для задания 2 представлен на рисунке 4.

```
a = int(input())
b = int(input())

c = 0  # сумма чисел
d = 1 # произведение чисел

for i in range(a, b + 1): # i – счетчик
    c += i
    d *= i

print(f"Сумма: {c}")
print(f"Произведение: {d}")
```

Рисунок 4 – Текст программы для задания 2

Программа выполняет итерацию по числовому диапозону с помощью цикла for и функции range (a, b + 1), накапливая промежуточные результаты

Результат работы программы при вводе 2 и 5 представлен на рисунке 5

```
Сумма: 14
Произведение: 120
```

Рисунок 5 - Результат работы программы для задания 2

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.

1.3 Задание 3. Подсчет измерений

Пользователь вводит неотрицательное целое число n, затем n значений температуры как вещественные числа, каждое на отдельной строке. Необходимо подсчитать количество измерений ниже нуля, равных нулю и выше нуля. Если n равно 0, все три количества равны нулю.

Блок-схема алгоритма программы для задания 3 представлена на рисунке 6.

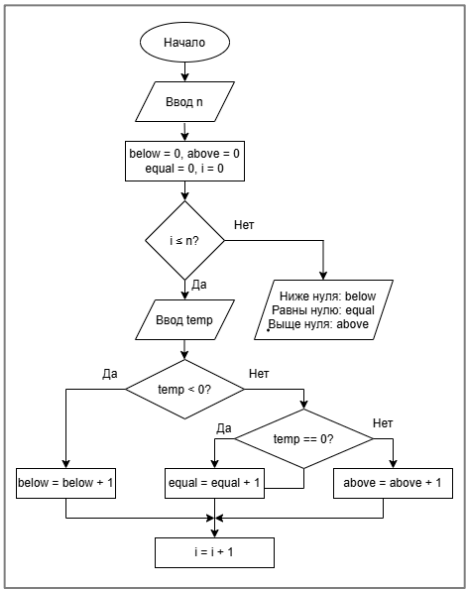


Рисунок 6 - Блок-схема алгоритма для задания 3

Текст программы для задания 3 представлен на рисунке 7.

```
n = int(input())
below = equal = above = 0

for _ in range(n):
    temp = float(input())
    if temp < 0: below += 1
    elif temp == 0: equal += 1
    else: above += 1

print(f"Ниже нуля: {below}")
print(f"Равны нулю: {equal}")
print(f"Выше нуля: {above}")
```

Рисунок 7 – Текст программы для задания 3

Программа считывает вещественные числа и последовательно обновляет счетчики групп.

Результат работы программы при вводе 5 и последующих значений -2,5, 0, -3.1, -1, 0 Представлен на рисунке 8.

```
Ниже нуля: 2
Равны нулю: 2
Выше нуля: 1
```

Рисунок 8 - Результат работы программы для задания 3

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.

1.4 Задание 4. Таблица умножения

Пользователь вводит целое число n от 1 до 9 включительно. Необходимо вывести таблицу умножения этого числа на целые числа от 1 до 10 включительно в порядке возрастания второго множителя.

Блок-схема алгоритма программы для задания 4 представлена на рисунке 9.

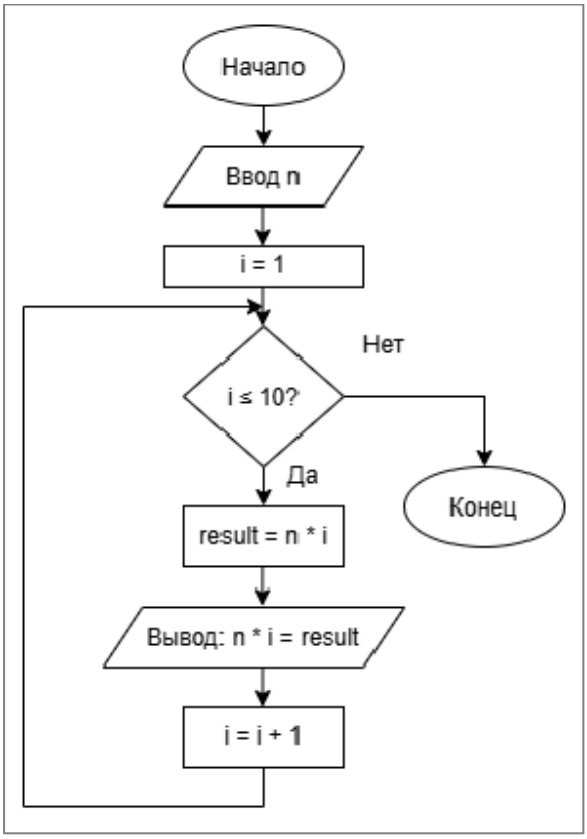


Рисунок 9 - Блок-схема алгоритма для задания 4

Текст программы для задания 4 представлен на рисунке 10.

```
n = int(input())

for i in range(1, 11):
    print(f"{n} * {i} = {n * i}")
```

Рисунок 10 – Текст программы для задания 4

Для вывода отформатированных строк используется цикл for с диапозоном range (1, 11 и f-строки)

Результат работы программы при вводе 3 представлен на рисунке 11.

```
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

Рисунок 11 - Результат работы программы для задания 4

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.

1.5 Задание 5. Учет покупок

Пользователь последовательно вводит стоимость каждой покупки в целых рублях на отдельной строке. Количество покупок заранее неизвестно; ввод завершается строкой «stop». Необходимо вывести количество покупок, общую и среднюю стоимость (с двумя знаками после точки). Если «stop» введена до первой покупки, выводится «Нет покупок».

Блок-схема алгоритма программы для задания 5 представлена на рисунке 12.

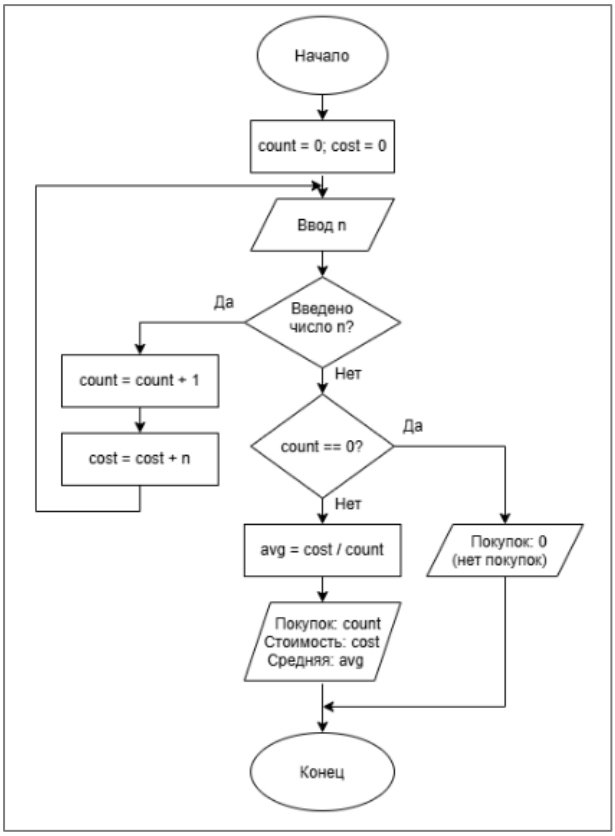


Рисунок 12 - Блок-схема алгоритма для задания 5

Текст программы для задания 5 представлен на рисунке 13.

```
count = 0
cost = 0
while True:
    try: n = int(input())
    except: break
    count += 1
    cost += n

print(f'Покупок: {count}')
print(f'Общая стоимость: {cost}')
print(f'Средняя стоимость: {cost / count}')
```

Рисунок 13 – Текст программы для задания 5

Цикл while True обрабатывает ввод до прерывания, перехватываемого блоком try..except, после чего рассчитываются и выводятся статистика и среднее значение.

Результат работы программы при вводе 120, 85, 200 и stop представлен на рисунке 14.

```
Покупок: 3
Общая стоимость: 405
Средняя стоимость: 135.0
```

Рисунок 14 - Результат работы программы для задания 5

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.

1.6 Задание 6. Треугольник Паскаля

Пользователь вводит количество строк n как целое число от 1 до 15 включительно. Необходимо вывести первые n строк треугольника Паскаля, начиная с вершины, используя вложенные циклы и целочисленные вычисления без готовых функций биномиальных коэффициентов и факториалов. Числа в строке разделяются одним пробелом. После треугольника выводится количество нечётных элементов во всех выведенных строках..

Блок-схема алгоритма программы для задания 6 представлена на рисунке 15.

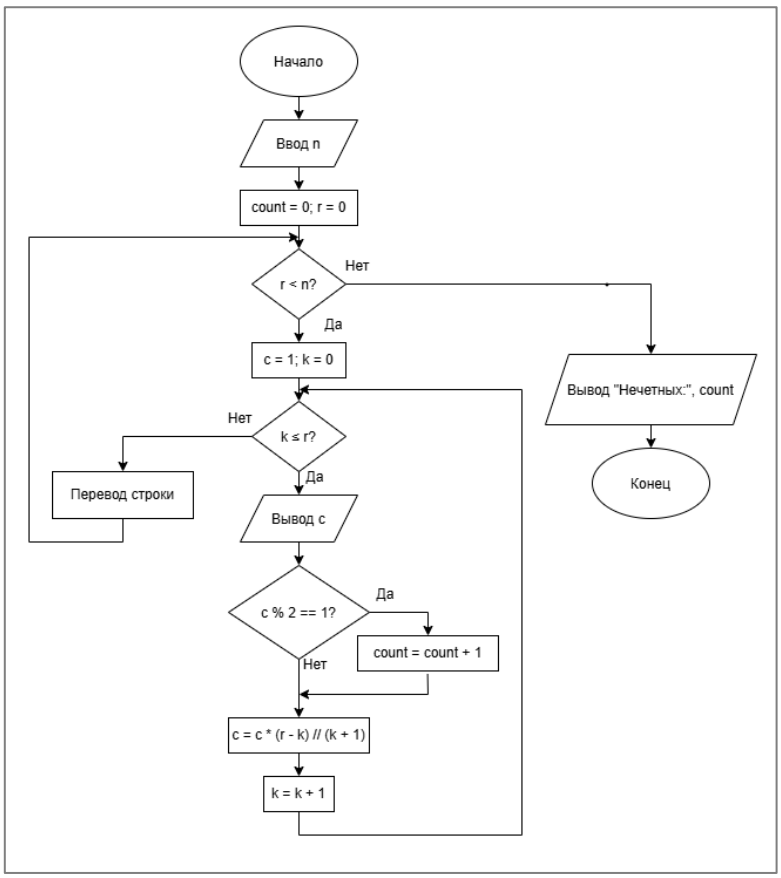


Рисунок 15 - Блок-схема программы для задания 6

Текст программы для задания 6 представлен на рисунке 16

```
n = int(input())
count = 0
for r in range(n):
    c = 1
    for k in range(r + 1):
        print(c, end=" ")
        if c % 2 == 1:
            count += 1
        c = c * (r - k) // (k + 1)
    print()
print("Нечетных элементов:", count)
```

Рисунок 16 - Результат программы для задания 6

Программа использует формулу для вычисления элементов строки без прямого подсчета факториалов, проверяя каждый элемент на нечетность.

Результат работы программы при вводе 5 представлен на рисунке 17.



Рисунок 17 - Результат работы программы для задания 6

Приведённый пример подтверждает, что программа корректно обрабатывает заданные входные данные и выводит результат в требуемом формате.